

HRMS

Employee Module

Employee core data, supporting records, Foundation relationships, APIs, business rules
and authorization

Technical implementation reference

1. Scope

This document describes the Employee module as implemented in the HRMS modular-monolith, including employee core data, supporting records, Foundation relationships, validation, authorization, and persistence conventions.

2. Architecture and Request Flow

HRMS follows a modular-monolith architecture. The API project is the composition root. Business use cases are implemented with MediatR commands/queries and handlers. Domain entities contain domain concepts and invariants. Infrastructure contains persistence, security and external implementation details. Shared BuildingBlocks provide abstractions such as IUserContext, IReadRepository and IWriteRepository. SQL Server is the persistence store and EF Core migrations manage schema evolution.

```
HTTP Request
↓
EmployeeController
↓
MediatR Command / Query
↓
Validation Behavior + Command Validation
↓
Handler / Business Rules
↓
IReadRepository / IWriteRepository
↓
EF Core / SQL Server
```

3. Employee Data Model

Area	Representative SQL tables	Purpose
Employee core	Employees	Primary employee record and employment classification/status reference
Personal	EmployeePersonalInformation	Personal/demographic information and related Foundation reference
Address	EmployeeAddresses	Employee addresses; references AddressType, Country and State
Emergency contact	EmployeeEmergencyContacts	Emergency contacts and Relationship reference
Bank	EmployeeBankAccounts	Employee banking/payment information
Government identifiers	EmployeeGovernmentIdentifiers	Identifiers linked to IdentifierType
Documents	EmployeeDocuments	Employee documents linked to DocumentType
Education	EmployeeEducations	Academic/educational history
Experience	EmployeeExperiences	Prior employment/work experience
Skills	EmployeeSkills	Employee-to-Skill relationship
Certifications	EmployeeCertifications	Employee-to-Certification relationship
Languages	EmployeeLanguages	Employee-to-Language relationship
Dependents	EmployeeDependents	Dependent records and Relationship reference where applicable
Nominees	EmployeeNominees	Nominee records and relationship information

4. SQL / Relational Design

Employees is the aggregate root for employee-owned records. Supporting tables should carry an EmployeeId foreign key. Foundation foreign keys should be used instead of duplicating master text. Many-to-many concepts such as employee skills, certifications and languages are represented by association records.

Field	Typical SQL type	Purpose
Id	uniqueidentifier	Primary key
CreatedBy	uniqueidentifier NULL	User that created the row
CreatedOn	datetimeoffset	Creation timestamp
ModifiedBy	uniqueidentifier NULL	Last modifying user
ModifiedOn	datetimeoffset NULL	Last modification timestamp
DeletedBy	uniqueidentifier NULL	User that soft-deleted the row
DeletedOn	datetimeoffset NULL	Soft-delete timestamp
IsDeleted	bit	Soft-delete flag

Important relational rules

EmployeeId foreign keys should be indexed. Association records should prevent duplicate active links. Soft-deleted records should be excluded from normal reads. Sensitive fields such as bank and identity information require controlled access and should not be exposed in unrelated list responses.

5. API Contract

Operation	HTTP	Endpoint pattern	Authorization
Get all	GET	/api/Employee	Employee.View
Get by ID	GET	/api/Employee/{id}	Employee.View
Create	POST	/api/Employee	Employee.Create
Update	PUT	/api/Employee/{id}	Employee.Update
Delete	DELETE	/api/Employee/{id}	Employee.Delete

Get employees

```
GET /api/Employee
Authorization: Bearer

200 OK
[
{
  "id": "xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxx",
  "employeeCode": "EMP001",
  "firstName": "Anzar",
  "lastName": "Ahmed",
  "fullName": "Anzar Ahmed",
  "email": "anzar.ahmed@hrms.com"
}
]
```

Get by ID

```
GET /api/Employee/{id}

200 OK
{
  "id": "xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxx",
  "employeeCode": "EMP001",
  "firstName": "Anzar",
  "lastName": "Ahmed"
}
```

Create

```
POST /api/Employee
Content-Type: application/json
```

```
{
  "employeeCode": "EMP001",
  "firstName": "Anzar",
  "lastName": "Ahmed",
  "email": "anzar.ahmed@hrms.com"
}
```

Update

```
PUT /api/Employee/{id}
Content-Type: application/json

{
  "id": "xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxx",
  "firstName": "Anzar",
  "lastName": "Ahmed"
}

204 No Content
```

Delete

```
DELETE /api/Employee/{id}

204 No Content
```

6. Business Rules

1. Route ID and command ID must match for update.
2. EmployeeCode should be unique within the business scope defined by the domain.
3. Required employee fields must be validated before persistence.
4. Foundation references must exist and be valid.
5. Related records must belong to the target employee.
6. Delete/deactivation must respect dependent data and audit requirements.
7. Sensitive employee data should be returned only from endpoints authorized for that purpose.
8. Query handlers should use projections/DTOs and avoid returning entity graphs unnecessarily.

7. Authorization

```
[Authorize(Policy = "Employee.View")]
GET /api/Employee

[Authorize(Policy = "Employee.Create")]
POST /api/Employee

[Authorize(Policy = "Employee.Update")]
PUT /api/Employee/{id}

[Authorize(Policy = "Employee.Delete")]
DELETE /api/Employee/{id}
```

The policy name is also the permission name. Authorization resolves the current user through `IUserContext`, loads `UserRoles`, resolves active Roles and RolePermissions, then verifies an active Permission with the required name.

8. Developer Change Guide

For a new employee sub-record: define the domain entity and FK, configure EF mapping/indexes, add DTO/command/query/validator/handler, use repository abstractions, add the API endpoint, apply the correct permission, update the migration and add integration tests. Avoid placing business rules in controllers.

9. Testing Checklist

Create valid/invalid employee; duplicate EmployeeCode; get all; get by ID/not found; update with matching and mismatching IDs; delete; invalid Foundation FK; unauthorized 401; authenticated but missing permission 403; authorized 200/201/204; verify soft-delete and audit behavior.